NBA STATLINE PREDICTOR

Machine Learning System Architecture & Development Roadmap

Player Predictions · Team Projections · Betting Lines · Fantasy Scoring

Version 1.0 | March 2026

Executive Summary

This document outlines the full architecture, technology stack, data strategy, modeling approach, and phased build plan for an NBA statistical prediction system. The system ingests historical and real-time NBA data, engineers predictive features, trains machine learning models, and surfaces actionable predictions across four use cases: player statline projection, team performance projection, sports betting over/under probabilities, and fantasy sports scoring.

System Capabilities at a Glance

- Predict individual player box scores (PTS, REB, AST, STL, BLK, TO, 3PM, FG%) for any given matchup
- Predict team-level game totals (total points, pace, FG%, 3P%, assists, turnovers)
- Generate over/under probability distributions for comparison against Vegas lines
- Compute fantasy scoring projections across DraftKings, FanDuel, and Yahoo formats
- Update predictions in near real-time as rosters, injuries, and lines shift daily

Recommended Technology Stack

The stack below is recommended based on performance benchmarks for tabular sports data, the maturity of available NBA data libraries, and the ability to scale from local development to a production API.

Layer	Technology	Purpose
Data Ingestion	nba_api (Python)	Official NBA stats endpoint wrapper — free, no key needed
Data Storage	PostgreSQL + TimescaleDB	Time-series optimized relational DB for game logs
Data Processing	pandas + polars	pandas for exploration; polars for production speed
Feature Engineering	scikit-learn Pipelines	Reproducible, pipeline-safe transformations
ML Modeling	XGBoost + LightGBM	Best-in-class for tabular regression; fast, interpretable
Probabilistic Output	MAPIE / quantile regression	Produces confidence intervals for betting use cases
Experiment Tracking	MLflow	Tracks model versions, params, and metrics
API Layer	FastAPI (Python)	Async REST API to serve predictions to frontend or apps
Scheduling	Apache Airflow	Orchestrates daily data pulls and model retraining

Layer	Technology	Purpose
Frontend Dashboard	React + Recharts	Interactive UI displaying predictions and line comparisons
Deployment	Docker + Railway/Render	Containerized, low-cost cloud deployment

Why XGBoost over Deep Learning: NBA stat prediction is a tabular regression problem. Gradient boosted trees consistently outperform neural networks on small-to-medium tabular datasets (< 1M rows). Deep learning provides no meaningful accuracy advantage here and adds significant complexity. XGBoost also offers native feature importance, which is critical for debugging and trust.

Data Architecture

Data Sources

All data is sourced from free, publicly accessible endpoints. No paid data subscription is required to build a production-quality system.

Source	What It Provides
nba_api (Python)	Box scores, game logs, play-by-play, player/team stats going back to 1996
Basketball-Reference (scrape)	Advanced stats: VORP, BPM, True Shooting %, usage rates
ESPN / CBS Scores API	Injury reports updated daily; lineup confirmations
Odds API (free tier)	Live and historical Vegas lines for O/U comparison
RotoWire / FantasyPros (scrape)	DFS salary data, projected ownership, expert rankings

Database Schema (Core Tables)

Table	Key Columns
players	player_id, name, team_id, position, height, weight, age
teams	team_id, name, conference, division, arena
games	game_id, date, home_team_id, away_team_id, season, is_playoffs
player_game_logs	game_id, player_id, min, pts, reb, ast, stl, blk, to, fgm, fga, fg3m, fg3a, ftm, fta
team_game_logs	game_id, team_id, pts, pace, off_rtg, def_rtg, ts_pct, ast, to, reb
injuries	player_id, date, status (GTD/OUT), description
betting_lines	game_id, sportsbook, player_prop, line, timestamp
predictions	game_id, player_id, model_version, predicted_pts, predicted_reb, ...

Feature Engineering

Feature engineering is the most impactful step in the entire pipeline. The model will only be as good as the signals fed into it. Below are the feature categories, organized by signal type.

Player Rolling Averages

- Last 5, 10, 20 game rolling averages for every stat (PTS, REB, AST, etc.)
- Rolling averages split by home vs. away games
- Rolling averages against top-10, top-20, bottom-10 defenses
- Season-to-date averages and trend slope (improving vs. declining)

Usage & Role Features

- Usage rate (USG%): percent of team possessions used while on court
- Minute share: average minutes / 48
- Assist opportunity rate: how often teammates are assisted by this player
- Lineup combinations: who is on court with the player (starter vs. bench context)

Opponent Defensive Features

- Opponent's Defensive Rating (points allowed per 100 possessions)
- Opponent's points allowed to each position (PG, SG, SF, PF, C) — last 15 games
- Opponent's pace factor: fast vs. slow tempo affects total fantasy opportunities
- Opponent's 3-point attempt rate allowed: key for shooter projections
- Opponent's steal rate and block rate: affects turnovers and interior scoring

Situational & Context Features

- Rest days since last game (0 = back-to-back, 3+ = well-rested)
- Home vs. away indicator
- Altitude indicator (Denver games affect pace and fatigue)
- Days into the season (fatigue, load management risk increases late in season)
- Game importance / playoff implications (affects star minutes)

Injury & Roster Context

- Teammate availability: star player out → usage spike for secondary players
- Opponent key player out: affects their defensive scheme
- Player's own recent injury history and minutes restrictions

> **Critical Feature: Opportunity Score**
>
> The single most predictive feature is projected minutes × usage rate. A player

> who averages 32 minutes at 28% usage on a healthy team will score far fewer points when a teammate is out and minutes jump to 38 at 35% usage. Always recalculate opportunity score at prediction time using current injury news.

Model Architecture

Model Structure: One Model Per Stat Per Context

Rather than a single model predicting everything, the system uses a collection of specialized models. This enables better accuracy and easier debugging.

Model	Predicts
PlayerPointsModel	Individual player points per game (continuous regression)
PlayerReboundsModel	Individual player total rebounds per game
PlayerAssistsModel	Individual player assists per game
PlayerThreesModel	3-pointers made per game
PlayerBonusModel	Steals, blocks, turnovers (low-frequency — Poisson regression)
TeamTotalsModel	Combined team points (game total) — key for O/U
TeamPaceModel	Possessions per game — upstream input to other models
TeamDefRatingModel	Opponent's defensive rating — used as feature in player models
FantasyScoreModel	Composite fantasy score (DK, FD, Yahoo) — aggregated from above

Model Type Recommendations

- **Primary model for all point/rebound/assist predictions. Handles non-linear interactions, missing values, and noisy sports data well.:** XGBoost Regressor
- **Faster alternative to XGBoost for real-time serving. Use in production API when latency matters.:** LightGBM
- **Used to generate 10th/50th/90th percentile predictions — essential for over/under probability distributions.:** Quantile Regression (XGBoost with quantile loss)
- **Better suited for low-count stats like steals (0–3 per game) and blocks where a normal distribution assumption fails.:** Poisson Regression
- **In Phase 3: blend XGBoost + LightGBM + historical median with a meta-learner for 2-4% accuracy gains.:** Ensemble / Stacking

Training & Validation Strategy

Sports data has a critical property: it is time-ordered. Standard k-fold cross-validation will leak future data into training, producing falsely optimistic accuracy scores. Always use time-series cross-validation.

Time-Series Walk-Forward Validation

Split: Train on seasons 2015–2022 → Validate on 2022–23 → Test on 2023–24
Never train on data after the validation cutoff date.
Use expanding window CV: each fold adds one more season of training data.
Evaluate on MAE (Mean Absolute Error) — e.g., ±4.2 points is meaningful context.
Compare model MAE vs. Vegas line accuracy as the gold-standard benchmark.

Target Metrics for Production Readiness

Stat	Target MAE (Mean Absolute Error)
Points	≤ 5.0 pts (Vegas lines hit ~4.5 pts MAE — aim to match)
Rebounds	≤ 2.5 reb
Assists	≤ 2.0 ast
3-Pointers Made	≤ 1.2 3PM
Team Game Total	≤ 8.0 pts (comparable to closing-line accuracy)
Fantasy Score (DK)	≤ 8.0 DK points

Betting & Fantasy Integration

Over/Under Probability Output

For each player prop and game total, the system will output a full probability distribution, not just a point estimate. This is the core value-add over simply reading the Vegas line.

- Use quantile regression to estimate P10, P25, P50, P75, P90 for each stat
- Convert the distribution to a probability of going over any given line: P(PTS > 24.5)
- Calculate implied edge vs. sportsbook: if model gives 62% and book implies 52%, that is +EV
- Flag high-confidence picks: model confidence interval does not overlap the line

Fantasy Score Computation

Platform	Scoring Formula
DraftKings	PTS×1 + REB×1.25 + AST×1.5 + STL×2 + BLK×2 + TO×(–0.5) + 3PM×0.5 + DD×1.5 + TD×3
FanDuel	PTS×1 + REB×1.2 + AST×1.5 + STL×2 + BLK×2 + TO×(–1)
Yahoo	PTS×1 + REB×1.2 + AST×1.5 + STL×2 + BLK×2 + TO×(–1) + 3PM×0.5

Projected fantasy score = apply the platform's formula to the predicted stat vector. Simulate 1,000 stat combinations using the prediction distribution (Monte Carlo) to get a fantasy score distribution with floor/ceiling projections — critical for GPP tournament lineup building.

Phased Development Roadmap

Phase	Name	Duration	Key Deliverable
Phase 1	Data Infrastructure & Historical Backfill	3–4 weeks	PostgreSQL DB with 5+ seasons of box scores
Phase 2	Feature Engineering Pipeline	2–3 weeks	Reproducible feature matrix generation script
Phase 3	Baseline ML Models	3–4 weeks	First XGBoost models; MAE benchmarked vs. Vegas
Phase 4	Prediction API	2 weeks	FastAPI endpoint returning JSON predictions
Phase 5	Betting & Fantasy Modules	2–3 weeks	O/U probabilities, DK/FD/Yahoo fantasy scores
Phase 6	Dashboard UI	2–3 weeks	React dashboard with today's games and predictions
Phase 7	Automation & Monitoring	2 weeks	Daily Airflow pipeline; model drift monitoring
Phase 8	Model Ensemble & Tuning	Ongoing	Stacking, hyperparameter search, edge tracking

Phase 1: Data Infrastructure

- Install and configure nba_api; write async game log fetcher
- Backfill 2015–2025 player and team game logs into PostgreSQL
- Set up TimescaleDB extension for efficient time-series queries
- Build daily incremental ingest job (only pull yesterday's games)
- Integrate Odds API for historical and live betting lines
- Deliverable: run a SQL query returning any player's last 30 game logs in < 100ms

Phase 2: Feature Engineering Pipeline

- Build rolling window function: compute 5/10/20-game averages from raw logs
- Build opponent defensive profile: def_rtg and position-specific allowed stats
- Build injury context feature: binary flags for teammate/opponent absences
- Wrap everything in a scikit-learn Pipeline for reproducible train/predict parity
- Deliverable: generate_features(game_id, player_id) returns a 60+ feature vector

Phase 3: Baseline Models

- Train XGBoost regressors for PTS, REB, AST, 3PM for the top 150 players by minutes
- Run time-series walk-forward validation across 2022–24 seasons
- Log experiments to MLflow: features used, hyperparams, MAE per stat
- Benchmark model MAE vs. Vegas implied accuracy on the same test set
- Deliverable: model achieves PTS MAE ≤ 5.5 on 2023–24 holdout

Phase 4: Prediction API

- Build FastAPI service with endpoints: /predict/player, /predict/team, /predict/game
- Endpoint accepts game_id and returns full predicted statline with confidence intervals
- Integrate injury feed: auto-adjust predictions when player status changes
- Add Redis caching: predictions cached for 15 minutes, refreshed on injury update
- Deliverable: API returns predictions in < 500ms for any player/game combination

Phases 5–8 Summary

- **Build over/under probability module; compute fantasy scores for all three platforms; add salary value ranking (projected points per $1,000 salary):** Phase 5
- **React dashboard: today's slate, predicted statlines, O/U comparison vs. Vegas, top fantasy value plays, confidence tier labels (High / Medium / Low):** Phase 6
- **Airflow DAGs for daily data pull at 8 AM and lineup-lock predictions at 6 PM; MLflow model drift monitoring alerts when MAE degrades > 15%:** Phase 7
- **Hyperparameter tuning with Optuna; ensemble stacking; track model edge vs. closing line value over time; retrain monthly with fresh season data:** Phase 8

Key Risks & Mitigations

Risk	Mitigation
Injury news arrives after lineup lock	Pull injury reports at 6 PM ET; allow manual override in API
Player traded mid-season changes team context	Rebuild team context features from trade date forward; flag first 10 games post-trade as uncertain
Model degrades as player ages / role changes	Retrain monthly; weight recent games 3x vs. older games in training
Small sample size for new players / rookies	Fall back to position-average model; flag predictions with low-confidence warning
Overfitting to specific defenses	Ensure opponent features use rolling averages, not single-game values
API rate limits on nba_api	Implement exponential backoff; cache raw responses in S3

Risk	Mitigation
Responsible gambling	Display confidence intervals prominently; add responsible gambling disclaimer to dashboard

Quick Start: First 30 Days

If you want to validate the concept before building the full system, here is the fastest path to a working prototype:

- **Day 1–3:** pip install nba_api pandas xgboost scikit-learn. Pull LeBron James last 3 seasons of game logs.
- **Day 4–7:** Engineer 10 rolling features. Train a simple XGBoost to predict PTS. Measure MAE.
- **Day 8–14:** Expand to top 50 players. Add opponent defensive rating as a feature. Compare MAE improvement.
- **Day 15–21:** Wrap predictions in a FastAPI endpoint. Run predictions for tonight's slate. Compare to Vegas.
- **Day 22–30:** Add REB + AST models. Compute DraftKings fantasy scores. You now have an MVP system.

Validation Benchmark: Beat the Public

The first milestone worth celebrating is NOT beating Vegas — it is beating public consensus. NumberFire and FantasyPros publish crowd-sourced projections. If your model's MAE is lower than theirs on a 2-week holdout, your feature engineering is working. Vegas accuracy is the long-term target and requires more data, sharper features, and live-line integration.